

# FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Department of Artificial Intelligence & Data Science · Karachi Campus

---

## PROGRAMMING FOR ARTIFICIAL INTELLIGENCE

*Laboratory Manual*

### LAB 01

#### Introduction to Python

Environment Setup · Jupyter Notebook · Variables · Expressions & Operators  
Data Types · Decision Making · Loops

PREPARED BY

**Khadeejah Ashraf**

Lab Instructor | Department of Artificial Intelligence & Data Science

## Contents

---

1. Lab Objectives
2. Lab Rules and Conduct
3. Getting Started with Python
4. Setting Up the Environment: Jupyter Notebook
5. Your First Python Program
6. Quotations and Comments
7. Variables and Assignment
8. Expressions, Statements, Operands and Operators
9. Standard Data Types
10. Decision Making
11. Loops
12. Quick Reference
13. Class Tasks (to be completed during the session)
14. Home Tasks
15. Submission and Assessment

## 1. Lab Objectives

---

By the end of this session, you should be able to:

- Explain what makes Python an interpreted, interactive and object-oriented language.
- Install Python and set up a working development environment based on **Jupyter Notebook**.
- Create, run, save and export a notebook, and explain the difference between a code cell and a markdown cell.
- Write, save and execute a simple Python program, both in a notebook and as a `.py` script.
- Distinguish between a **statement** and an **expression**, and identify the **operands** and **operators** inside an expression.
- Use arithmetic, assignment, comparison, logical, membership and identity operators, and apply the rules of operator precedence.
- Declare variables and use the five standard data types: numbers, strings, lists, tuples and dictionaries.
- Apply indexing and slicing to strings, lists and tuples.
- Control the flow of a program using `if`, `elif` and `else` statements.
- Repeat a block of statements using `while` and `for` loops, and alter that repetition with `break` and `continue`.
- Solve small problems independently by combining all of the above.

## 2. Lab Rules and Conduct

---

- Maintain discipline throughout the lab session.
- Listen carefully and follow the instructions as they are given.
- Raise your hand if you face any problem; do not leave your seat.
- **Class tasks** must be completed and demonstrated during the session itself.
- **Home tasks** must be submitted before the deadline announced on the course portal.
- Get your work checked by the instructor before the end of the session.

**Reminder:** work that is not checked during the session carries no marks, even if the code runs correctly.

### 3. Getting Started with Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. It is designed to be highly readable: it uses English keywords where many other languages use punctuation, and it has fewer syntactical constructions than most other languages.

#### 3.1 Key Characteristics

| Characteristic    | What it means in practice                                                                                                             |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Interpreted       | Python is processed at run time by the interpreter. You do not need to compile your program before executing it.                      |
| Interactive       | You can sit at a Python prompt and interact with the interpreter directly, testing one statement at a time.                           |
| Object-Oriented   | Python supports the object-oriented style of programming, which encapsulates code within objects.                                     |
| Dynamically Typed | A variable takes the type of whatever you assign to it, and that type can change during the program.                                  |
| Beginner-Friendly | Python supports everything from simple text processing to web applications, data science and AI, so the same language grows with you. |

#### 3.2 Installing Python

Two installation routes are available; either one is acceptable for this course.

**Route A — Anaconda Distribution (recommended).** Download the Anaconda installer for your operating system from [anaconda.com](https://anaconda.com), run it, and accept the defaults. Anaconda ships Python together with Jupyter Notebook, NumPy, Pandas, Matplotlib and the other scientific libraries you will need later in this course, so nothing else has to be installed by hand.

**Route B — Official Python installer.** Download Python 3.x from [python.org](https://python.org), and on Windows tick “**Add Python to PATH**” on the very first screen of the installer. Jupyter is then installed separately from the command line:

```
python --version      # verify the installation
pip install notebook  # install Jupyter Notebook
pip install jupyterlab # optional: the newer JupyterLab interface
```

**Check:** open a terminal (Command Prompt on Windows, Terminal on macOS/Linux) and type `python --version`. If a version number such as `Python 3.12.4` is printed, the installation succeeded. If instead you see “command not found”, Python was not added to PATH.

#### 3.3 Three Ways to Run Your Code

| Method            | How to start it                                                    | Best used for                                              |
|-------------------|--------------------------------------------------------------------|------------------------------------------------------------|
| Interactive shell | Type <code>python</code> in the terminal                           | Testing a single statement or checking a result quickly    |
| Script file       | Write <code>program.py</code> , run <code>python program.py</code> | Complete programs and all home-task submissions            |
| Jupyter Notebook  | Type <code>jupyter notebook</code> in the terminal                 | Learning, experimenting, and mixing code with explanations |

All three run exactly the same language. This lab is conducted in Jupyter Notebook because it lets you keep your notes, your code and your output side by side; the home tasks, however, must be submitted as `.py` script files.

## 4. Setting Up the Environment: Jupyter Notebook

A Jupyter Notebook is an interactive document made of **cells**. A cell is either a piece of Python code that you can run, or a block of formatted text that explains what the code does. The file is saved with the `.ipynb` extension.

### 4.1 Launching Jupyter

Open a terminal, move to the folder where you want your work to be saved, and launch the server:

```
cd Desktop/PAI_Lab01      # move to your working folder
jupyter notebook          # start the server
```

A browser tab opens at `http://localhost:8888` showing the contents of that folder. Choose **New ► Python 3** to create a fresh notebook, and rename it by clicking on the word **Untitled** at the top.

**Do not close the terminal.** the black terminal window is the notebook **server**. Closing it stops the kernel and your cells will no longer run. Shut the server down properly with `Ctrl + C` when you are finished.

### 4.2 Anatomy of the Notebook Interface

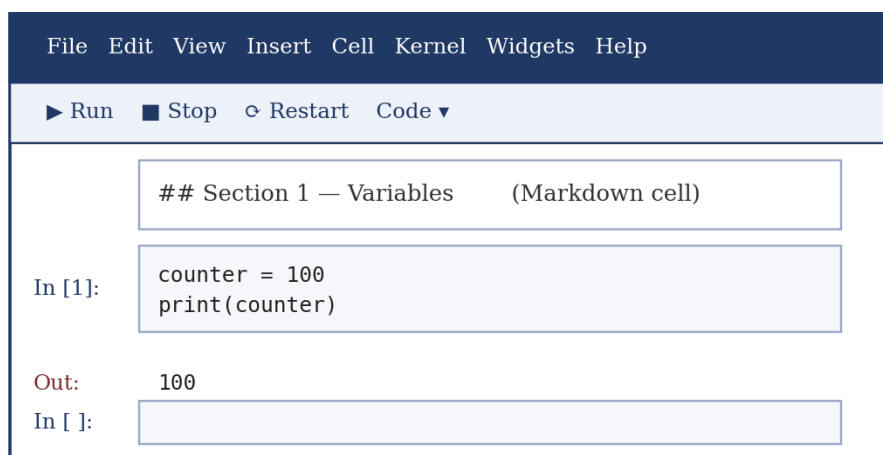


Figure 4.1 The main elements of the Jupyter Notebook interface

### 4.3 Code Cells and Markdown Cells

The cell type is chosen from the toolbar dropdown, or with the keyboard shortcuts `Y` (code) and `M` (markdown) while the cell is selected but not being edited.

| Cell type | Contains                                                                                            | What happens when you run it                                                                           |
|-----------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Code      | Python statements                                                                                   | The code is executed by the kernel and the result appears directly underneath as <code>Out[ ]</code> . |
| Markdown  | Formatted text: headings ( <code>#</code> ), <b>bold</b> ( <code>**text**</code> ), lists, formulas | The text is rendered into a neat heading or paragraph — nothing is executed.                           |

Use markdown cells generously. A notebook that carries a short heading and one line of explanation above every code cell is far easier to read — and to mark — than a notebook that is nothing but code.

### 4.4 Essential Keyboard Shortcuts

A cell is in **edit mode** (green border) when you are typing inside it, and in **command mode** (blue border) after you press Esc. The shortcuts below work in command mode.

| Shortcut            | Action                                          |
|---------------------|-------------------------------------------------|
| Shift + Enter       | Run the current cell and move to the next one   |
| Ctrl + Enter        | Run the current cell and stay on it             |
| Alt + Enter         | Run the current cell and insert a new one below |
| A / B               | Insert a new cell above / below                 |
| D D (press D twice) | Delete the selected cell                        |
| Y / M               | Convert the cell to code / markdown             |
| Z                   | Undo the deletion of a cell                     |
| Esc / Enter         | Leave edit mode / enter edit mode               |

## 4.5 The Kernel and the Execution Order

The **kernel** is the Python process that actually runs your code. It remembers every variable you have created, and cells are executed in the order in which **you** run them — not in the order in which they appear on the screen. The number inside In [ ]: tells you that order.

```
In [1]: x = 5
In [3]: print(x * 2)      # runs fine – the kernel still remembers x
In [2]: x = 100          # ...but this ran BEFORE the line above
```

The three commands you will need most sit under the **Kernel** menu:

- **Interrupt** — stops a cell that is taking too long or is stuck in an infinite loop (the same as Ctrl + C).
- **Restart** — clears every variable from memory and gives you a clean interpreter.
- **Restart & Run All** — restarts, then runs every cell from top to bottom. Always do this before submitting, to prove that your notebook works when read from the beginning.

## 4.6 Saving and Exporting

Ctrl + S saves the notebook as an .ipynb file. To convert your work into the script format required for submission, use **File ► Download as ► Python (.py)**, or run the following in the terminal:

```
jupyter nbconvert --to script Lab01.ipynb # produces Lab01.py
```

## 4.7 Common Pitfalls

| Symptom                                        | Cause and cure                                                                                         |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| NameError: name 'x' is not defined             | The cell that creates x has not been run yet, or the kernel was restarted. Run the earlier cell first. |
| The cell shows In [*] and never finishes       | The code is still running, or it is an infinite loop. Use <b>Kernel ► Interrupt</b> .                  |
| A stale result that no longer matches the code | Cells were run out of order. Use <b>Kernel ► Restart &amp; Run All</b> .                               |
| input() seems to hang                          | Jupyter is waiting for you to type into the small box under the cell and press Enter.                  |

## 5. Your First Python Program

The `print()` function displays a value on the screen. In Python 3, the parentheses are required. (In the very old Python 2.4, `print` was a statement and the parentheses could be omitted — you will still see that style in old books, but it is an error today.)

```
print("Hello, Python!")
```

*Output*

```
Hello, Python!
```

**Note:** Python is case-sensitive. `Print("Hi")` and `PRINT("Hi")` will both fail; only `print` is correct.

## 6. Quotations and Comments

### 6.1 Quotation Marks

Python accepts single (`'`), double (`"`) and triple (`' '` or `"""`) quotes to denote string literals, as long as the same type of quote starts and ends the string. Triple quotes allow a string to span multiple lines. All three of the following are legal:

```
word = 'word'
sentence = "This is a sentence."
paragraph = """This is a paragraph.
It is made up of multiple lines
and sentences."""
```

### 6.2 Comments

A hash sign (`#`) that is not inside a string literal begins a comment. Every character after the `#` up to the end of that physical line is part of the comment, and the interpreter ignores it.

```
# This is a full-line comment
print("Hello, Python!") # this part is ignored too
```

**Good practice:** write a short comment above every exercise explaining what the program does. In a notebook, put that explanation in a markdown cell instead. Comments are part of your lab marks.

## 7. Variables and Assignment

Python variables do not need an explicit declaration to reserve memory space; the declaration happens automatically when you assign a value. The equal sign (`=`) is the assignment operator: the operand on its left is the **name** of the variable, and the operand on its right is the **value** stored in it.

```
counter = 100      # an integer assignment
miles    = 1000.0  # a floating point number
name     = "Usman" # a string

print(counter)
print(miles)
print(name)
```

*Output*

```
100
1000.0
Usman
```

## 7.1 Rules for Naming Variables

- A name may contain letters, digits and underscores, but must not begin with a digit.
- Names are case-sensitive: `total`, `Total` and `TOTAL` are three different variables.
- Reserved keywords (`if`, `else`, `for`, `while`, `True`, `None`, ...) cannot be used as names.
- Choose meaningful names — `total_marks` tells the reader far more than `t`.

## 7.2 Multiple Assignment

Python lets you assign several variables in one statement, which is handy for swapping values.

```
a, b = 10, 20
a, b = b, a          # swap without a temporary variable
print(a, b)
```

*Output*

```
20 10
```

# 8. Expressions, Statements, Operands and Operators

Everything you write in Python is built from a small number of ideas. Getting the vocabulary right now will make the rest of the course much easier to follow.

## 8.1 Statements

A **statement** is a complete instruction that Python can execute. An assignment is a statement, and so is a `print()` call, an `if`, or a `for` loop. A statement performs an action; on its own it does not produce a value.

```
x = 12                # assignment statement
print(x)              # a call statement
if x > 10:             # a compound statement (header + indented body)
    print("large")
```

## 8.2 Expressions

An **expression** is any combination of values, variables, operators and function calls that Python can **evaluate** into a single value. `3 + 4` is an expression whose value is 7; `x > 10` is an expression whose value is `True`. Every expression can be used wherever a value is expected — which is why an expression can sit on the right-hand side of an assignment, but never on the left.

```
3 + 4 * 2             # expression    -> 11
length * width        # expression    -> a number
x > 10 and x < 20      # expression    -> True or False
area = length * width # statement containing an expression
```

**Rule of thumb:** if you can print it, it is an expression. `print(3 + 4)` works; `print(x = 5)` does not, because an assignment is a statement and has no value.

## 8.3 Operands and Operators

Inside an expression, an **operator** is the symbol that says what to do, and the **operands** are the values it acts upon. In `7 + 3`, the `+` is the operator while 7 and 3 are the operands.

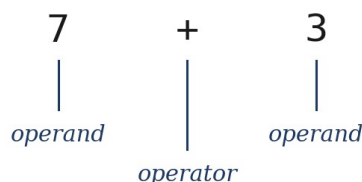


Figure 8.1 The parts of a simple expression

An operator that needs two operands (like `+`) is called **binary**; one that needs a single operand (like the negation in `-x`, or `not flag`) is called **unary**.

## 8.4 Arithmetic Operators

| Operator        | Meaning                                | Example (a = 17, b = 5) | Result |
|-----------------|----------------------------------------|-------------------------|--------|
| <code>+</code>  | Addition                               | <code>a + b</code>      | 22     |
| <code>-</code>  | Subtraction                            | <code>a - b</code>      | 12     |
| <code>*</code>  | Multiplication                         | <code>a * b</code>      | 85     |
| <code>/</code>  | Division — always gives a float        | <code>a / b</code>      | 3.4    |
| <code>//</code> | Floor division — discards the fraction | <code>a // b</code>     | 3      |
| <code>%</code>  | Modulus — the remainder                | <code>a % b</code>      | 2      |
| <code>**</code> | Exponentiation                         | <code>b ** 2</code>     | 25     |

**Why % matters:** the remainder is how you test divisibility. `n % 2 == 0` is true for even numbers, and `n % 3 == 0` is true for multiples of three — the whole idea behind the FizzBuzz task later in this manual.

## 8.5 Assignment Operators

The compound assignment operators combine an arithmetic operation with an assignment, so that `count = count + 1` can be written more briefly.

| Operator                | Example             | Equivalent to          |
|-------------------------|---------------------|------------------------|
| <code>=</code>          | <code>x = 5</code>  | assigns 5 to x         |
| <code>+=</code>         | <code>x += 3</code> | <code>x = x + 3</code> |
| <code>-=</code>         | <code>x -= 3</code> | <code>x = x - 3</code> |
| <code>*=</code>         | <code>x *= 3</code> | <code>x = x * 3</code> |
| <code>/=</code>         | <code>x /= 3</code> | <code>x = x / 3</code> |
| <code>//= %= **=</code> | <code>x %= 3</code> | <code>x = x % 3</code> |

## 8.6 Comparison Operators

These compare two operands and always evaluate to `True` or `False`. They are the expressions you place inside an `if` or a `while`.

| Operator                              | Meaning                         | Example (a = 10, b = 20)       |
|---------------------------------------|---------------------------------|--------------------------------|
| <code>==</code>                       | Equal to                        | <code>a == b</code> → False    |
| <code>!=</code>                       | Not equal to                    | <code>a != b</code> → True     |
| <code>&gt;</code> <code>&lt;</code>   | Greater than, less than         | <code>a &lt; b</code> → True   |
| <code>&gt;=</code> <code>&lt;=</code> | Greater / less than or equal to | <code>a &gt;= 10</code> → True |

**A classic mistake:** = assigns a value while == compares two values. Writing `if x = 5:` is a `SyntaxError`.

## 8.7 Logical Operators

Logical operators combine conditions. Python also treats `0`, `0.0`, `""`, `[]`, `{}` and `None` as false, and everything else as true.

| Operator | Meaning                                     | Example (a = 10, b = 20)                  |
|----------|---------------------------------------------|-------------------------------------------|
| and      | True only if <b>both</b> operands are true  | <code>a &lt; b and b &gt; 5</code> → True |
| or       | True if <b>at least one</b> operand is true | <code>a &gt; 50 or b == 20</code> → True  |
| not      | Reverses the result                         | <code>not (a == 10)</code> → False        |

## 8.8 Membership and Identity Operators

| Operator    | Meaning                                                        | Example                                |
|-------------|----------------------------------------------------------------|----------------------------------------|
| in          | True if the value is found in the sequence                     | <code>'a' in 'Pakistan'</code> → True  |
| not in      | True if the value is <b>not</b> found                          | <code>5 not in [1, 2, 3]</code> → True |
| is / is not | Compares identity — whether two names refer to the same object | <code>x is None</code>                 |

## 8.9 Operator Precedence

When an expression contains several operators, Python evaluates them in a fixed order. The table runs from the highest precedence to the lowest; operators on the same row are evaluated from left to right.

| Order | Operators           | Description                                       |
|-------|---------------------|---------------------------------------------------|
| 1     | ( )                 | Parentheses — evaluated first, always             |
| 2     | **                  | Exponentiation                                    |
| 3     | +x -x not x         | Unary plus, minus and logical NOT                 |
| 4     | * / // %            | Multiplication, division, floor division, modulus |
| 5     | + -                 | Addition and subtraction                          |
| 6     | < <= > >= == !=     | Comparisons                                       |
| 7     | in not in is is not | Membership and identity                           |
| 8     | and                 | Logical AND                                       |
| 9     | or                  | Logical OR                                        |

```
print(2 + 3 * 4)      # 14, not 20  -> * binds tighter than +
print((2 + 3) * 4)    # 20          -> parentheses win
print(2 ** 3 ** 2)    # 512         -> ** groups right to left
print(10 - 4 - 3)     # 3           -> - groups left to right
print(not True and False) # False   -> not is applied first
```

**Advice:** even when you know the precedence rules, add parentheses whenever an expression is long. Code is read far more often than it is written.

## 8.10 Type Conversion

`input()` always returns a string, so text typed by the user must be converted before it can take part in arithmetic. This is called **type casting**.

```
age_text = input("Enter your age: ") # "20" -> a string
age = int(age_text)                  # 20 -> an integer
print(age + 1)

print(type(age_text))                # check a value's type
print(float("3.5"), int(3.99), str(42))
```

*Output*

```
21
<class 'str'>
3.5 3 42
```

**Careful:** `int()` truncates rather than rounds — `int(3.99)` is 3. And `"5" + 3` raises a `TypeError`, because Python will not silently mix a string with a number.

## 9. Standard Data Types

Data stored in memory can be of many types. A person's age is stored as a numeric value, while an address is stored as alphanumeric characters. Python defines standard data types that determine both the storage method and the operations possible on the value. The five standard types are:

| Data Type  | Written as                 | Ordered? | Changeable? |
|------------|----------------------------|----------|-------------|
| Numbers    | <code>x = 10</code>        | —        | —           |
| String     | <code>s = 'Hello'</code>   | Yes      | No          |
| List       | <code>L = [1, 2, 3]</code> | Yes      | Yes         |
| Tuple      | <code>T = (1, 2, 3)</code> | Yes      | No          |
| Dictionary | <code>D = {'a': 1}</code>  | By key   | Yes         |

### 9.1 Numbers

Number data types store numeric values. A number object is created the moment you assign a value to it.

```
var1 = 1          # int
var2 = 10.5       # float
print(var1 + var2)
```

*Output*

```
11.5
```

### 9.2 Strings

Strings are a contiguous set of characters written inside quotation marks. Subsets of a string can be taken with the slice operator (`[ ]` and `[ : ]`), with indexes starting at 0 at the beginning of the string and working backwards from -1 at the end.

```

text = 'Hello World!'
print(text)          # prints the complete string
print(text[0])       # prints the first character
print(text[2:5])     # characters from the 3rd to the 5th
print(text[2:])      # everything from the 3rd character onwards
print(text * 2)      # prints the string two times
print(text + "TEST") # prints a concatenated string

```

**Output**

```

Hello World!
H
llo
llo World!
Hello World!Hello World!
Hello World!TEST

```

**Understanding slicing.** In `text[a:b]` the character at index `a` is included but the character at index `b` is not. This is why `text[2:5]` gives three characters, not four.

| Character | H   | e   | l   | l  | o  |    | W  | o  | r  | l  | d  | !  |
|-----------|-----|-----|-----|----|----|----|----|----|----|----|----|----|
| Index     | 0   | 1   | 2   | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| Negative  | -12 | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

**Caution:** `str` is the name of a built-in Python type. Using it as a variable name works, but hides the built-in for the rest of the program. Prefer a name such as `text` or `message`.

### 9.3 Lists

The list is the most versatile data type available in Python. It is written as a list of comma-separated values (items) between square brackets. Importantly, the items in a list need not be of the same type.

```

list1 = ['physics', 'chemistry', 1997, 2000]
list2 = [1, 2, 3, 4, 5, 6, 7]
print("list1[0]: ", list1[0])
print("list2[1:5]: ", list2[1:5])

```

**Output**

```

list1[0]: physics
list2[1:5]: [2, 3, 4, 5]

```

**Updating a list.** You can update a single element or a slice of elements by placing the index on the left-hand side of the assignment operator, and you can add a new element to the end with the `append()` method.

```

subjects = ['physics', 'chemistry', 1997, 2000]
print("Value available at index 2 : ", subjects[2])
subjects[2] = 2001
print("New value available at index 2 : ", subjects[2])
subjects.append('maths')      # adds a new item at the end
print(subjects)

```

**Output**

```

Value available at index 2 : 1997
New value available at index 2 : 2001
['physics', 'chemistry', 2001, 2000, 'maths']

```

**Deleting list elements.** Use the `del` statement when you know exactly which index you are deleting, and the `remove()` method when you know the value but not its position.

```
subjects = ['physics', 'chemistry', 1997, 2000]
print(subjects)
del subjects[2]          # delete by index
print("After deleting value at index 2 : ", subjects)
subjects.remove('chemistry') # delete by value
print(subjects)
```

#### Output

```
['physics', 'chemistry', 1997, 2000]
After deleting value at index 2 : ['physics', 'chemistry', 2000]
['physics', 2000]
```

| Operation                                        | Meaning                                                  |
|--------------------------------------------------|----------------------------------------------------------|
| <code>len(L)</code>                              | Number of items in the list                              |
| <code>L.append(x)</code>                         | Adds <code>x</code> to the end of the list               |
| <code>L.insert(i, x)</code>                      | Inserts <code>x</code> at position <code>i</code>        |
| <code>L.remove(x)</code>                         | Removes the first occurrence of the value <code>x</code> |
| <code>del L[i]</code>                            | Removes the item at index <code>i</code>                 |
| <code>L.sort()</code> / <code>L.reverse()</code> | Sorts / reverses the list in place                       |
| <code>max(L)</code> / <code>min(L)</code>        | Largest / smallest item                                  |
| <code>sum(L)</code>                              | Sum of all items (numeric lists only)                    |

## 9.4 Tuples

A tuple is another sequence data type that is similar to a list. A tuple consists of a number of values separated by commas, but unlike lists it is enclosed within parentheses. The main difference is that the elements and size of a list can be changed, whereas a tuple cannot be updated. Tuples can therefore be thought of as read-only lists.

```
record = ('abcd', 786, 2.23, 'john', 70.2)
tinytuple = (123, 'john')
print(record)          # prints the complete tuple
print(record[0])        # prints the first element
print(record[1:3])      # elements from the 2nd till the 3rd
print(record[2:])        # elements from the 3rd onwards
print(tinytuple * 2)     # prints the tuple two times
print(record + tinytuple) # prints a concatenated tuple
```

#### Output

```
('abcd', 786, 2.23, 'john', 70.2)
abcd
(786, 2.23)
(2.23, 'john', 70.2)
(123, 'john', 123, 'john')
('abcd', 786, 2.23, 'john', 70.2, 123, 'john')
```

The following code is invalid, because it attempts to update a tuple. The same operation on a list is perfectly valid:

```
record = ('abcd', 786, 2.23, 'john', 70.2)
items = ['abcd', 786, 2.23, 'john', 70.2]
record[2] = 1000    # INVALID – tuples cannot be modified
```

```
items[2] = 1000    # valid – lists can be modified
```

**Output**

```
TypeError: 'tuple' object does not support item assignment
```

|                | List                         | Tuple                       |
|----------------|------------------------------|-----------------------------|
| Written with   | Square brackets [ ]          | Parentheses ( )             |
| Can be changed | Yes                          | No                          |
| Typical use    | Data that grows or is edited | Fixed records and constants |

## 9.5 Dictionaries

Python's dictionaries are a kind of hash table. They consist of key–value pairs. A key can be almost any Python type but is usually a number or a string, while a value can be any arbitrary Python object. Dictionaries are enclosed by curly braces, and values are assigned and accessed using square brackets.

```
data = {}
data['one'] = "This is one"
data[2] = "This is two"

tinydict = {'name': 'john', 'code': 6734, 'dept': 'sales'}
print(data['one'])    # value for the 'one' key
print(data[2])        # value for the 2 key
print(tinydict)       # the complete dictionary
print(tinydict.keys()) # all the keys
print(tinydict.values()) # all the values
```

**Output**

```
This is one
This is two
{'name': 'john', 'code': 6734, 'dept': 'sales'}
dict_keys(['name', 'code', 'dept'])
dict_values(['john', 6734, 'sales'])
```

**Note:** from Python 3.7 onwards a dictionary preserves the order in which keys were inserted, so the output above appears in the same order the pairs were written.

| Operation         | Meaning                                     |
|-------------------|---------------------------------------------|
| D[key]            | Reads the value stored against key          |
| D[key] = value    | Adds a new pair, or updates an existing key |
| D.keys()          | All the keys in the dictionary              |
| D.values()        | All the values in the dictionary            |
| D.items()         | All key–value pairs, useful for looping     |
| key in D          | Tests whether a key exists                  |
| max(D, key=D.get) | The key that holds the largest value        |

## 10. Decision Making

Decision making is the anticipation of conditions occurring during the execution of a program, and the specification of actions to be taken according to those conditions. A decision structure evaluates an expression that produces TRUE or FALSE as its outcome, and you decide which statements should execute in each case.

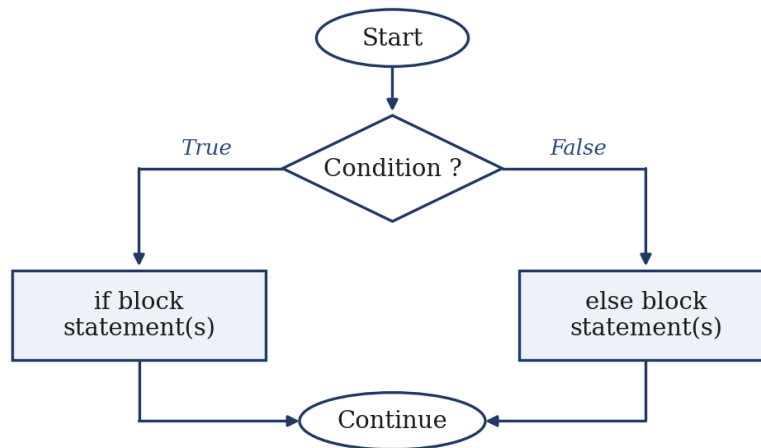


Figure 10.1 General form of a decision-making structure

## 10.1 The if Statement

The `if` statement contains a logical expression; the data is compared and a decision is made based on the result of that comparison. The general form is:

```
if expression:
    statement(s)
```

**Indentation is part of the syntax.** The indented block below the colon is the body of the `if`. Use four spaces consistently — mixing tabs and spaces causes an `IndentationError`.

```
var1 = 100
if var1:
    print("1 - Got a true expression value")
    print(var1)

var2 = 0
if var2:
    print("2 - Got a true expression value")
    print(var2)

print("Good bye!")
```

*Output*

```
1 - Got a true expression value
100
Good bye!
```

**Why?** any non-zero number is treated as true and 0 as false, so the second block never executes.

## 10.2 The if...else Statement

An `else` statement can be combined with an `if` statement. It contains a block of code that executes when the conditional expression in the `if` statement resolves to 0 or FALSE. The `else` part is optional, and there can be at most one `else` following an `if`.

```
if expression:
    statement(s)
else:
```

```

    statement(s)

amount = int(input("Enter amount: "))

if amount < 1000:
    discount = amount * 0.05
    print("Discount", discount)
else:
    discount = amount * 0.10
    print("Discount", discount)

print("Net payable:", amount - discount)

```

**Output**

```

Enter amount: 600
Discount 30.0
Net payable: 570.0

```

**Note:** `input()` always returns a string, so it is wrapped in `int()` to convert the text typed by the user into a number.

### 10.3 The `if...elif...else` Statement

The `elif` statement allows you to check multiple expressions for `TRUE` and to execute a block of code as soon as one of the conditions evaluates to `TRUE`. Like `else`, `elif` is optional; but unlike `else`, of which there can be at most one, an `if` may be followed by any number of `elif` statements.

```

if expression1:
    statement(s)
elif expression2:
    statement(s)
elif expression3:
    statement(s)
else:
    statement(s)

amount = int(input("Enter amount: "))

if amount < 1000:
    discount = amount * 0.05
    print("Discount", discount)
elif amount < 5000:
    discount = amount * 0.10
    print("Discount", discount)
else:
    discount = amount * 0.15
    print("Discount", discount)

print("Net payable:", amount - discount)

```

**Output**

```

Enter amount: 3000
Discount 300.0
Net payable: 2700.0

```

**Nested decisions.** An `if` may itself contain another `if`. Keep the nesting shallow — two levels is usually enough, and a combined condition written with `and` is often clearer than a nested block.

```

marks = 85
if marks >= 50:
    if marks >= 80:

```

```

        print("Distinction")
    else:
        print("Pass")
else:
    print("Fail")

```

## 11. Loops

In general, statements are executed sequentially: the first statement in a function is executed first, then the second, and so on. There may be a situation where you need to execute a block of code several times. A loop statement allows us to execute a statement, or a group of statements, multiple times.

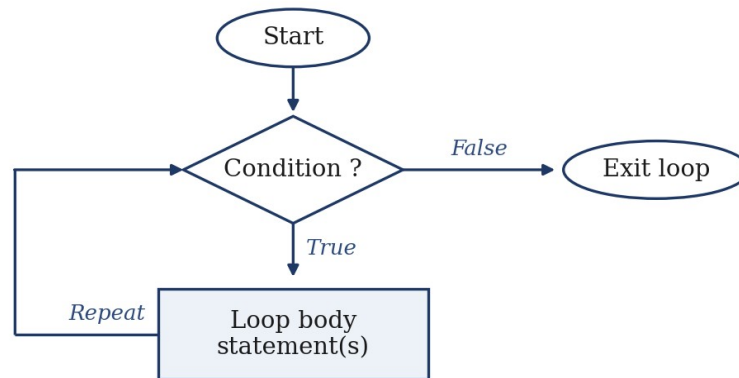


Figure 11.1 General form of a loop structure

### 11.1 The while Loop

A while loop repeatedly executes a target statement as long as a given condition remains true. The condition is tested before every pass, so if it is false at the start the body never runs at all.

```

count = 0
while count < 9:
    print("The count is:", count)
    count = count + 1

print("Good bye!")

```

#### Output

```

The count is: 0
The count is: 1
...
The count is: 8
Good bye!

```

**Careful:** something inside the loop must eventually make the condition false. Forgetting `count = count + 1` produces an infinite loop; press `Ctrl + C` to stop a runaway program, or use **Kernel ► Interrupt** in Jupyter.

### 11.2 The for Loop

The `for` statement has the ability to iterate over the items of any sequence, such as a list or a string, taking one item at a time.

```

for var in range(5):           # 0, 1, 2, 3, 4
    print(var)

for letter in 'Python':       # traversal of a string

```

```
print('Current Letter :', letter)

fruits = ['banana', 'apple', 'mango']
for fruit in fruits:      # traversal of a list
    print('Current fruit :', fruit)

print("Good bye!")
```

**Output**

```
0 1 2 3 4
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : h
Current Letter : o
Current Letter : n
Current fruit : banana
Current fruit : apple
Current fruit : mango
Good bye!
```

**The range() function.** range(n) produces the numbers 0 to n-1; range(a, b) produces a to b-1; and range(a, b, step) counts in jumps of step. For example, range(1, 11) is exactly what you need for a multiplication table.

| Use a...   | When...                                       | Example                |
|------------|-----------------------------------------------|------------------------|
| for loop   | the number of repetitions is known in advance | for i in range(1, 11): |
| while loop | you repeat until a condition changes          | while count < 9:       |

**11.3 break, continue and Nested Loops**

break exits a loop immediately, and continue skips the rest of the current pass and moves on to the next item.

```
for n in range(1, 10):
    if n == 7:
        break          # stop the loop completely
    if n % 2 == 0:
        continue       # skip the even numbers
    print(n, end=' ')
```

**Output**

```
1 3 5
```

A loop placed inside another loop is a **nested loop**. The inner loop completes all of its passes for every single pass of the outer loop — the standard way to print a pattern.

```
for i in range(1, 5):
    for j in range(i):
        print("*", end="")
    print()
```

**Output**

```
*
**
***
****
```

## 12. Quick Reference

The following built-in functions will cover almost everything required in this lab.

| Function                       | Purpose                                     | Example                             |
|--------------------------------|---------------------------------------------|-------------------------------------|
| <code>print(x)</code>          | Displays a value on the screen              | <code>print("Hi", 5)</code>         |
| <code>input(msg)</code>        | Reads a line of text typed by the user      | <code>name = input("Name: ")</code> |
| <code>int(x) / float(x)</code> | Converts text into a whole / decimal number | <code>n = int(input("n: "))</code>  |
| <code>str(x)</code>            | Converts a number into text                 | <code>str(42) → '42'</code>         |
| <code>type(x)</code>           | Reports the data type of a value            | <code>type(3.5) → float</code>      |
| <code>len(x)</code>            | Number of items or characters               | <code>len([1, 2, 3]) → 3</code>     |
| <code>range(a, b)</code>       | Sequence of numbers from a to b-1           | <code>range(1, 11)</code>           |
| <code>sum / max / min</code>   | Total, largest and smallest of a list       | <code>max([4, 9, 2]) → 9</code>     |
| <code>sorted(L)</code>         | Returns a sorted copy of a sequence         | <code>sorted([3, 1, 2])</code>      |
| <code>x % y</code>             | Remainder — the key to testing multiples    | <code>12 % 3 == 0 → True</code>     |
| <code>x ** y</code>            | Exponentiation (x raised to the power y)    | <code>3 ** 2 → 9</code>             |

## 13. Class Tasks

These five tasks are to be completed **during the lab session** and demonstrated to the instructor before you leave. Each one deliberately combines several concepts from this manual, so read the whole question before you start writing code. Work in the accompanying notebook `Lab01_Introduction_to_Python.ipynb`, in the section titled **Class Tasks**.

**C1. Student Result Card — variables, operators and type conversion.** Take the student's name, roll number and the marks obtained in three subjects (each out of 100) as input from the user. Using arithmetic operators, calculate the total marks, the average and the percentage, and print a neatly formatted result card. The percentage must be printed rounded to two decimal places. Finally, use the modulus operator to print whether the total is an even or an odd number.

Hint: `input()` returns a string, so every mark must be wrapped in `int()`. Use `round(value, 2)` for the percentage.

**C2. Expression Evaluator — operands, operators and precedence.** Take two integers `a` and `b` from the user, then print the result of every arithmetic operator (+, -, \*, /, //, %, \*\*) applied to them, followed by the result of every comparison operator (==, !=, >, <, >=, <=). Then, **before running your code**, write on paper the value you expect for each of the following expressions, and afterwards print them to check whether you were right:

- (i) `a + b * 2`      (ii) `(a + b) * 2`      (iii) `a ** 2 // b`  
 (iv) `a > b` and `a % 2 == 0`      (v) `not a == b` or `b > 100`

In a comment, explain in one line why (i) and (ii) give different answers.

**C3. String Inspector — indexing, slicing and loops.** Take a single word from the user and, without using any built-in reversing function, print all of the following: the length of the word; its first and last characters; the first three and the last three characters using slicing; every alternate character (`word[: :2]`); the word in reverse using a loop; and the number of vowels it contains. Use the membership operator `in` for the vowel test.

Sample: for the input `Pakistan` your program should report a length of 8, a reverse of `natsikaP` and 3 vowels.

- C4. Marks Dictionary — dictionaries, decisions and loops.** Ask the user how many subjects they wish to enter. Using a loop, read that many subject names together with their marks, and store them in a dictionary using the subject name as the key. Then, by looping over the dictionary, print: every subject with its marks and its grade (A for 80 and above, B for 70–79, C for 60–69, D for 50–59, otherwise F); the average marks; the name of the subject with the highest marks; and the name of the subject with the lowest marks.

Hint: for `subject, marks in record.items():` gives you both the key and the value in one loop.

- C5. Menu-Driven List Manager — the whole lab in one program.** Write a program that keeps a list of numbers and repeatedly shows the following menu inside a `while` loop until the user chooses to exit:

1. Add a number 2. Remove a number 3. Show the list 4. Show statistics 5. Exit

Option 4 must report the count of items, the sum, the average, the largest and smallest values, and how many of the numbers are even. Use `elif` to select the option, `break` to leave the loop, and print a sensible message when the user enters an invalid choice or tries to remove a value that is not in the list.

**Assessment of class tasks:** you must be able to explain any line of your code to the instructor. A working program that you cannot explain earns no marks.

## 14. Home Tasks

Attempt all the questions below. Write each program in a **separate .py file**, test it with more than one input, and submit it according to the naming convention in Section 15.

- Q1.** Calculate the body mass index (BMI) from two values input by the user, where  $BMI = \text{weight} / (\text{height})^2$ .
- Q2.** Write a program that works as a simple calculator, performing the four basic operations (+, −, \*, /) on two numbers input by the user. The following conditions must be satisfied: (a) a '+' sign must not be used for addition; (b) you may use a maximum of 3 variables — no more variables are allowed; (c) the program should ask the user which operation he/she wants to perform and give the output accordingly.
- Q3.** Write a program to take an integer list as input from the user, count all the even numbers in that list, and print the count.
- Q4.** Write a program that takes a list of numbers as input and returns the sum of all the elements in the list.
- Q5.** Write a program to take a list and a number as input from the user, and then delete all the elements in the list that are less than that number.
- Q6.** Aliza has got 40 marks in Physics, 78 in Chemistry and 82 in Maths. Take these marks as input from the user and store them in a dictionary with the subject name as the key and the marks as the value. By accessing the data stored in the dictionary, print the average of her marks and also print the name of the subject in which she scored highest.
- Q7.** Write a program that accepts a word from the user and reverses it using a loop. For example, 'Pakistan' becomes 'natsikaP'.
- Q8.** Write a Python program that iterates over the integers from 1 to 50. For multiples of three print “Fizz” instead of the number, for multiples of five print “Buzz”, and for numbers that are multiples of both three and five print “FizzBuzz”.

### Sample output for Q8

```
1
2
Fizz
4
Buzz
Fizz
7
...
```

14  
FizzBuzz

**Q9.** Write a Python program to create the multiplication table (from 1 to 10) of a number entered by the user.

**Sample output for Q9**

```
Input a number: 6
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
...
6 x 10 = 60
```

**Q10.** Write a Python program to get the largest number from a list input by the user.

**Q11.** Write a program to take the marks of 3 subjects as input from the user and store them in a dictionary with appropriate keys. Using that dictionary, calculate the average and the percentage of those marks.

**Q12.** Write a program to print a dictionary in which the keys are the numbers between 1 and 15 (both included) and the values are the squares of the keys.

## 15. Submission and Assessment

Follow the naming convention below so that your work can be identified and graded correctly:

- **Class tasks:** completed in the notebook `RollNo_Lab01_ClassTasks.ipynb` and demonstrated before the end of the session. Run **Kernel ► Restart & Run All** before you call the instructor.
- **Home tasks:** one script per question, named `RollNo_Lab01_Qn.py` — for example `22K-1234_Lab01_Q3.py`.
- Place all the files for this lab in a single folder and compress it before uploading.
- Every file must begin with a comment giving your name, roll number and the question number.
- Code that does not run will not be marked. Test your program before calling the instructor.

| Assessment Criterion                                          | Weight |
|---------------------------------------------------------------|--------|
| Correctness of output for all valid inputs                    | 40%    |
| Class tasks completed and demonstrated in the session         | 20%    |
| Correct use of data types, operators and control structures   | 20%    |
| Readability: meaningful variable names, indentation, comments | 10%    |
| In-lab demonstration and understanding of the code            | 10%    |

*End of Lab 01*